Introduzione

La spinta principale che portò alla creazione di Java non fu Internet ma la necessità di un linguaggio indipendente dalla piattaforma che potesse essere utilizzato per creare software da incorporare in vari dispositivi d'elettronica di consumo. Nel periodo in cui furono elaborati i dettagli di Java, cominciò ad emergere un secondo e più importante fattore che avrebbe svolto un ruolo fondamentale nel futuro di Java, ovviamente il WWW.
Java è un linguaggio di programmazione che consente ai Web browser di scaricare frammenti di codice e quindi di eseguirli localmente. Grazie alle sue caratteristiche fin dai primi giorni in cui è stato presentato (nel 1995) il linguaggio Java viene considerato la soluzione di molti problemi relativi ai sistemi client/server e al WWW. Prima della sua presentazione, un qualsiasi utente che si cimentasse nella navigazione sulla rete Internet, poteva trovare lungo il suo viaggio "solo" un insieme di documenti uniti insieme da collegamenti ipertestuali. Oggi, grazie a questi linguaggi, è possibile inserire all'interno di una pagina Web, anche veri e propri programmi, che possono essere scaricati da server remoti ed eseguiti direttamente sulla macchina locale.

Java è un linguaggio di programmazione ad oggetti (lo si può definire "puro" perché a differenza del C++ non consente una programmazione strutturata) ideato dalla JavaSoft, una divisione della Sun Microsystem. E' stato sviluppato in origine per gestire piccoli dispositivi elettronici, ma nel giro di pochi anni è diventato uno strumento di sviluppo estremamente diffuso per applicazioni in ambiente Internet ed Intranet, grazie alle possibilità grafiche che mette a disposizione degli sviluppatori e alla sua caratteristica d'essere multipiattaforma. Java può essere utilizzato per creare due tipi di programmi: applicazioni e applet.
Un applicazione è un programma che gira sul vostro computer sotto il sistema operativo di detto computer.
Un applet è un'applicazione concepita per essere trasmessa su Internet ed eseguita da un browser Web compatibile con Java. In pratica un applet è un piccolo programma Java dinamicamente caricato sulla rete, come un file.
La caratteristica essenziale di Java è che l'output di un compilatore Java non è un codice eseguibile, ma si tratta di un insieme altamente ottimizzato d'istruzioni (denominato *bytecode)* concepite per essere eseguite da una macchina virtuale emulata dal sistema runtime di Java.

Se l'impiego di pagine dal contenuto eseguibile, da un lato comporta un vantaggio per l'utente, in quanto permette un approccio molto più dinamico e interattivo con la Rete, dall'altro introduce nuovi problemi legati soprattutto alla sua sicurezza. I rischi stanno nel fatto che un programma scaricato dalla rete ed eseguito sulla macchina locale, ha in sé tutte le potenzialità per arrecare un danno all'utente e il rischio è tanto più grande quanto maggiore è la sua capacità d'accesso alle funzionalità della macchina stessa.

1 Caratteristiche di Java

Introduciamo ora le principali caratteristiche del linguaggio Java per poi rivolgerci ad aspetti e problematiche relative al modello di sicurezza. Per prima cosa bisogna dire che Java è un linguaggio semplice, multi-threaded, dinamico, robusto, distribuito, interpretato, sicuro, ad alte prestazioni, portabile. Inoltre, poiché vogliamo capire la relazione di Java con la sicurezza dei computer, dovremmo avere una profonda conoscenza del linguaggio.
Cominciamo con il discutere le sue peculiari caratteristiche.

Orientato agli oggetti: diversamente dal C++, Java è realmente object-oriented. I programmi di Java sono

composti da una o più classi le quali sono collezioni di oggetti di dati e metodi per poterli manipolare. Le classi sono organizzate gerarchicamente, nel senso che le sottoclassi ereditano metodi e strutture dalle loro superclassi.

Strongly typed: un programma Java non ha libero accesso alla memoria del computer, infatti l'accesso è limitato a specifiche aree controllate. La type security viene controllata dal bytecode verifier quando viene caricato nella Virtual Machine (JVM), la quale al runtime esegue ulteriori controlli di sicurezza.

Multi-threaded: un programma Java può eseguire più lavori nello stesso tempo. Per esempio un applet Java può leggere un file di musica e nello stesso tempo può scaricare un file. Poiché Java è multi-threaded, supporta l'esecuzione concorrente di molti processi, migliorando le performance multimediali.

Garbage collection: il garbage collection consiste nel tenere traccia dell'uso della memoria, cioè quando degli item non sono più necessari, la memoria utilizzata dagli stessi viene resa libera per altri usi. Java fornisce un garbage collector che utilizza un thread eseguito in background; quest'approccio consente di prevenire problemi dovuti ai dangling pointer (puntatori volanti).

Senza puntatori: questa è un'altra caratteristica del moderno schema della gestione della memoria di Java. Invece di permettere l'accesso alla memoria attraverso puntatori, la memoria è gestita attraverso riferimenti. Proprio perché i riferimenti non possono essere manipolati attraverso l'aritmetica (+, -, *, /, ecc…), vengono eliminati potenziali bug, rendendo Java un linguaggio più affidabile e sicuro.

Trattamento delle eccezioni: definisce come il programma gestisce le condizioni di errore. Il lancio e la cattura delle eccezioni consente di amministrare gli errori durante l'esecuzione che altrimenti manderebbero in crash il sistema.

Collegamento Dinamico: i moduli Software (le classi in Java) sono linkati insieme quando necessario (runtime) e non durante la fase di compilazione. Un problema può sorgere quando vi sono differenze operative tra versioni diverse di una stessa classe.

2 Il Problema della sicurezza

Gli applet Java sono molto utili nelle applicazioni Client/Server e in tutte le applicazioni che riguardano la rete. Il loro utilizzo però porta nuovi problemi che riguardano la sicurezza.

Il problema della sicurezza ovviamente non riguarda solo gli "addetti ai lavori", ma può interessare in maniera diversa alle varie tipologie di persone ovvero per ognuno di questi gruppi di persone, il problema sulla sicurezza ha delle caratteristiche differenti.

Possiamo considerare quattro tipologie di persone:

- Utenti Web.
- Programmatori Java.
- Amministratori di sistema.
- Businessman.

Utente Web: considerando il gran numero di persone che naviga su Internet questo problema non è da sottovalutare, infatti un Utente Web è inconsapevolmente anche un utente di Java. Conoscendo il modo in cui

opera Java, la sicurezza dei computer è una questione fondamentale, proprio perché navigando nel Web si scarica codice Java automaticamente che viene eseguito sulla macchina utente. E', quindi, importante limitare ciò che può compiere tale codice. Tramite Java è facile creare un programma che in apparenza faccia una cosa, ma che in realtà crea problemi al sistema dell'utente .

Programmatori Java: anche se Java offre strutture crittografiche e meccanismi di sicurezza a livello di linguaggio, programmatori inesperti o hacker potrebbero comunque realizzare applicazioni non sicure. Questo significa che gli sviluppatori di Java devono conoscere nel miglior modo possibile i problemi che riguardano la sicurezza, valutare i rischi nella fase di design e sviluppo e testarli attentamente.

Amministratore di sistema: ormai le LAN (Local Area Network) isolate appartengono al passato, oggi la maggior parte delle LAN sono direttamente collegate ad Internet. Gli amministratori di sistema, quindi, non possono trascurare la sicurezza. Il problema è che gli utenti vogliono Java mentre gli amministratori di sistema vorrebbero evitarlo per non prendere eccessivi rischi, questo è un classico esempio di compromesso tra funzionalità e sicurezza.

Businessman: un imprenditore si chiede soprattutto quali sono i rischi a cui va incontro utilizzando applicazioni Java, ma alcune delle compagnie che non usano Java pretendono che anche i loro clienti e i loro soci d'affari si privino di questa tecnologia.

I progettisti di Java sono consapevoli dei molti rischi associati al codice mobile. Java è stato progettato tenendo ben presente le problematiche della sicurezza, proprio per evitare ai comuni utenti di dover diventare esperti in sicurezza.

Nella forma standard, Java fornisce un approccio multistrato alla sicurezza. Gli strati superficiali includono

- Accesso limitato al file system e alla rete.
- Accesso limitato alle parti interne del browser.
- Una serie di controlli runtime e loadtime che verificano se il bytecode rispetta le regole.
- Un sistema per firmare il codice e per assegnare a quest'ultimo livelli di privilegi.

2.1 Il costo della sicurezza

La vera sicurezza è ottenuta rafforzando costantemente meccanismi di sicurezza e politiche di Informazione per adattarsi ai sempre nuovi problemi e attacchi, ma ci sono costi associati alle procedure di sicurezza, e queste procedure fanno sì che molti dei vantaggi di questo linguaggio possano essere ridotti. Le compagnie che vogliono fare della semplice pubblicità sul Web possono utilizzare un semplice firewall per scoraggiare i vandali elettronici. Nel caso di un grande istituto finanziario che ha che fare con molti milioni di lire è comprensibile l'uso di misure di protezione più sofisticati, come crittografie a chiave pubblica, reti private dedicate e metodi di sicurezza standard. Per applicazioni come il controllo del traffico aereo militare e sistemi intelligenti, il rischio di connessioni illecite a questi sistemi tramite Internet può superare i benefici che Internet stesso può dare.
Il costo delle implementazioni dei meccanismi di sicurezza è molto importante per due fattori. Se la nuova tecnologia risulta comoda e conveniente per ottenere lo stesso livello di sicurezza dei sistemi esistenti, essa sarebbe molto interessante. D'altra parte quest'aumento della sicurezza corrisponde ad un aumento dei costi, le organizzazioni devono esaminare i costi in previsione dei possibili rischi.Ogni volta che viene calcolato il costo della sicurezza, il riutilizzo di questi calcoli è un importante fattore. Se i meccanismi di sicurezza consumano troppo tempo oppure sono troppo difficili da usare, questi possono far diminuire la produttività. Gli utenti che trovano queste politiche di sicurezza difficili da capire o da applicare le possono ignorare o

implementarne qualcuna a caso.
Java è in grado di fornire dei meccanismi di sicurezza trasparenti, che non richiedono nessuna conoscenza di meccanismi da parte dell'utente. Questo è possibile perché il modello di sicurezza di Java ha avuto l'intenzione di proteggere l'utente finale da applet ostili e da sorgenti poco affidabili.

3 Modello di sicurezza in Java

Le problematiche relative alla sicurezza viste fin ora riguardano sia gli utenti Java che gli sviluppatori di Java. Usare Java è facile come entrare in rete, però essere un utente Java comporta dei rischi.

L'ambiente di sviluppo Java comprende tre componenti:

1. Un linguaggio di programmazione che compila in un formato intermedio e indipendente dall'architettura chiamato bytecode.
2. La Java Virtual Machine (JVM) che esegue il bytecode.
3. Un ambiente di esecuzione nella JVM e fornisce alcune classi base utili per costruire applicazioni complete.

Prima della discussione del nuovo modello utilizzato da Java 1.2 (capitolo 4), introdurremo il modello utilizzato fino alla versione Java 1.1 le cui parti principali continuano ad esser presenti nella evoluzione della versione 1.2 per mantenere una compatibilità verso il basso.
La limitazione principale del modello 1.1 consiste nella scarsa granularità ottenibile: o l'utente non può fare niente o può fare tutto. In tale modello, le azioni degli applet sono ristrette nelle "sandbox", un'area del browser Web dedicata agli applet. Gli applet possono fare tutto nelle sandbox, ma non possono leggere o scrivere i dati che stanno al di fuori di questa. La sandbox garantisce addirittura che se un utente scarica un applet malizioso, questo non danneggerà la macchina locale.
La sandbox è composta dai seguenti componenti che lavorano insieme:

- il Bytecode Verifier, assicura che solo legittimi bytecode siano eseguiti.
- il Class Loader, rende Java dinamico.
- il Security Manager, controlla l'accesso alle risorse cruciali del sistema.

La Figura 3.1 mostra come le parti che costituiscono la sandbox lavorano insieme per garantire l'esecuzione di codice inaffidabile in maniera sicura.

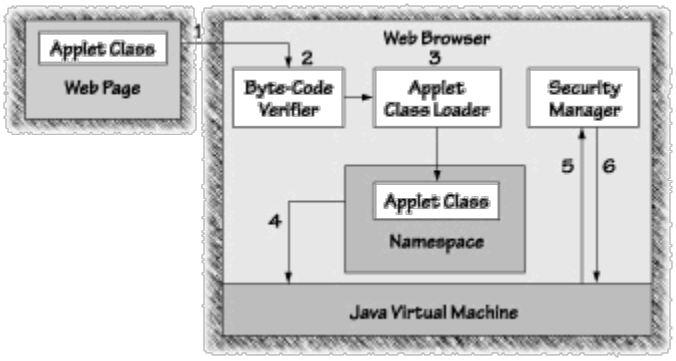

Figure 3.1 Come Java implementa l'originale approccio sandbox per il codice mobile. Il codice sorgente Java è compilato in un Java bytecode che è trasmesso

attraverso il Web al browser che lo richiede. L'HTML, in una pagina Web, specifica quale codice deve essere preso dal Web server. La richiesta del browser Web, indotta da un'azione quando un utente clicca su un hyperlink, (1) preleva il bytecode dal Web, (2) lo verifica, (3) lo istanzia in una classe o insieme di classi in un namespace. L'applet è eseguito e (4) quando invoca un metodo pericoloso (5) il Security Manager viene consultato prima che il metodo venga eseguito. Il Security Manager (6) esegue i runtime-checks sulla base dell'origine della classe chiamante e con l'autorità di porre un veto sull'operazione.

Il bytecode gira nella Java Virtual Machine, quindi può essere eseguito su tutte le piattaforme in cui vi è la JVM. Alcuni browser, come Netscape e Internet Explorer, hanno la propria versione della JVM incapsulata al loro interno (built-in), quindi se un utente accede ad una pagina HTML che include il tag <APPLET>, il browser esegue automaticamente l'applet Java indicato dal tag.

3.1 Il Bytecode Verifier

Quando un programma Java è compilato, il risultato è un codice indipendente dalla piattaforma (bytecode) che viene memorizzato in un file binario di uno specifico formato (class file). Il compito del verificatore è di controllare il class file e il suo contenuto. Il processo di verifica è diviso in due passi principali:

1. *Internal check*. Prima controlla che il formato del class file sia della forma corretta e successivamente verifica il bytecode analizzando il flusso dei dati.
2. *Runtime check*. Conferma l'esistenza e la compatibilità di classi, campi e metodi riferiti simbolicamente. Solo il codice che passa la verifica viene eseguito.

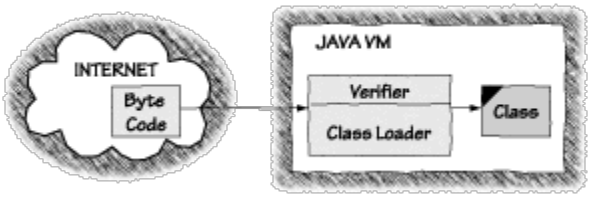

Fig. 3.2 Il bytecode verifier esamina attentamente il bytecode prima di essere eseguito sulla locale VM .Il verifier gioca un ruolo essenziale affrontando la sicurezza basata sul linguaggio Java, che è stato costruito sul concetto fondamentale della type safety.

3.1.1 Type Safety

Il linguaggio Java è stato progettato per far rispettare la Type Safety. Questo significa che i programmi che tentano di accedere alla memoria con modi inappropriati, non vengono soddisfatti. Più precisamente, Type Safety comporta che un programma non può eseguire operazioni su un oggetto Java a meno che tali operazioni sono valide per l'oggetto considerato.
Ogni oggetto Java è memorizzato in qualche regione della memoria del computer e viene etichettato con un class tag. Un semplice modo per gestire la Type Safety sarebbe quello di controllare il class tag di un oggetto prima di ogni operazione su tale oggetto(dynamic type checking), ma questa soluzione risulta inefficiente per il troppo tempo speso a controllare i class tag. Java usa, appena possibile, uno *static type checking* per migliorare le prestazioni: prima che un programma venga eseguito, Java cerca di capire quali controlli sui tag

siano non necessari (controlli che avranno sempre successo) e quali invece possano generare un errore (controlli che falliranno sempre). Il bytecode verifier è uno static type checker molto efficiente che permette di eliminare operazioni di controllo dei tag dai progammi Java rendendoli type safe ed abbastanza efficienti. Lo static type checking ha anche altri vantaggi, infatti potrebbe essere effettuato al tempo di compilazione in modo da comunicare allo sviluppatore eventuali errori prima che il codice venga utilizzato.

3.2 Il Class Loader

Uno dei principi centrali di Java è costruire codice realmente mobile. Il sistema richiede l'abilità di caricare dinamicamente codice proveniente dall'esterno. In Java, il codice è caricato (dal disco o dalla rete) per mezzo del Class Loader.
I Class Loader determinano quando e come le classi possono essere aggiunte all'ambiente Java in esecuzione. Parte del loro lavoro è di assicurarsi che parti fondamentali dell'ambiente runtime non siano rimpiazzati da codici impostori. Per esempio, la falsificazione della Security Manager non deve essere permessa.

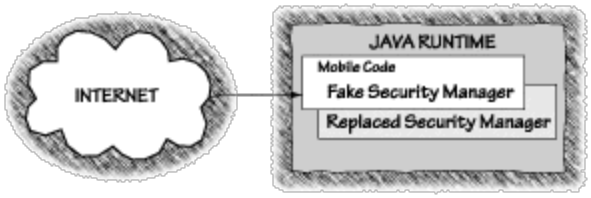

Fig. 3.3 Lo *spoofing* si presenta quando qualcuno o qualcosa pretende di essere qualcosa che non è. In questa figura, un classe esterna è arrivata da Internet e dichiara di essere il Security Manager (allo scopo di rimpiazzare il reale Security Manager). Se al codice esterno fosse permesso di fare questo, il sistema di sicurezza di Java sarebbe banalmente violato.

I class loader eseguono due funzioni:

1. Quando la VM ha bisogno di caricare il bytecode di una classe, essa chiede al class loader di trovarlo. Ciascun class loader può usare un suo proprio metodo di ricerca.
2. Il class loader definisce e gestisce il namespace.

Ci sono due varietà di class loader:

- Primordial Class Loader (PCL)
- Class Loader Objects (CLOs)

Esiste un solo Primordial Class Loader, che è parte essenziale della VM e che non può essere ignorato. Viene invocato durante il bootstrapping dell'ambiente Java e carica le classi fidate, di solito dal disco locale.

I CLOs sono oggetti come ogni altro oggetto scritto in Java, compilato in un byte code, e caricato dalla VM (con l'aiuto di qualche altro class loader). Questi CL forniscono a Java le capacità di dynamic linking. La VM, per default, tratta le classi caricate dai CLOs come non fidate. Ci sono tre tipi di CLOs definiti in JDK:

- Applet Class Loaders (ACLs)

- RMI Class Loaders (RMICLs)
- Secure Class Loaders (SCLs)

La figura 3.4 mostra la gerarchia ereditaria dei class loader disponibile in Java 2.

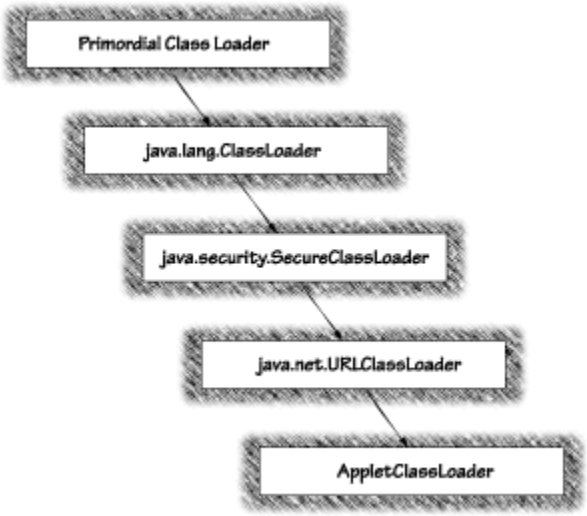

Fig. 3.4 I Class Loader forniscono la capacità del dynamic loading.

3.3 Il Security Manager

Il Security Manager (SM) è un singolo oggetto Java che esegue controlli durante l'esecuzione dei metodi pericolosi. Il codice nella libreria Java consulta il SM ogni volta che un'operazione potenzialmente rischiosa è richiesta. In questo modo vengono incapsulate le risorse che potrebbero essere usate illecitamente dal codice mobile. Il SM prende la decisione finale se l'operazione deve essere permessa o respinta, in quest'ultimo caso una Exception Security viene lanciata. Le decisioni della SM sono prese secondo l'origine della classe che ha richiesto l'operazione. Ovviamente alle classi built-in sono concessi maggiori privilegi rispetto a quelle caricate dalla rete.

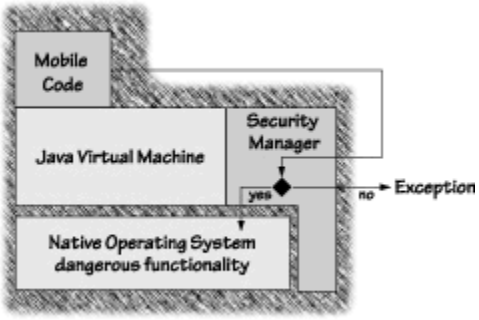

Fig. 3.5 Il Security Manager salvaguardia classificando le chiamate potenzialmente pericolose al sistema operativo locale (che sta sotto la VM). In questo modo, Java può incapsulare le risorse che sarebbero, altrimenti, abusate dal codice mobile.